Code2Test: Intent-First, Execution-Verified Test Generation for Legacy Python Code

Code2Test Contributors
https://github.com/Cirkitly/code2test

October 2026

Abstract

Large language models (LLMs) can write unit tests for untested legacy code, but practical generators keep only tests that pass, and recent studies show that this filter locks existing bugs into the generated suite. We present Code2Test, an open-source tool for Python that first infers, stores and lets developers edit an explicit statement of intent for every function and class. It generates pytest tests against that intent, executes them, repairs failures from their pytest output, and diagnoses tests that still fail as a wrong test, a code bug or a wrong intent. Suspected code bugs are kept as expected failures with the diagnosis attached, so the written suite always passes while the signal survives. A built-in benchmark reports mutation score and the accuracy of the diagnosis on failures of known cause. On two open-source libraries (426 and 2,170 lines) with two MiniMax models, Code2Test produced passing tests for 96%–100% of components. Before any repair, 85%–99% of generated tests passed, and the suites killed 59%–78% of sampled mutants. The diagnosis was correct on 97%–100% of failures of known cause. Surviving mutants concentrated in one function that a run tested only structurally and in string literals that tests do not observe, and nearly all intents came from docstrings. Because both libraries are mature, the study measures validity and fault detection, not whether intent-first generation avoids locking in bugs; we describe the experiment that would. We also report eight failure modes that appeared only with a real model. Each of the tool's techniques has a precedent; its contribution is their combination in one open tool and the empirical lessons from running it.

1 Introduction

Much production code has no tests, and writing them for legacy code is slow because the intended behavior must first be reconstructed. LLMs can write unit tests [2, 10, 11], but three problems stand between an LLM and a suite a team can merge.

No oracle. A test needs an expected value. When the expected value is taken from the code alone, the test can only confirm what the code already does. Mathews and Nagappan [7] showed the consequence for tools that keep only passing tests. On 287 buggy Python programs, 59.6% (Cover-Agent) and 68.1% (CoverUp) of the final suites passed on the buggy code but failed on the correct one. Zhao et al. [13] call this the *misguidance* effect of buggy code.

Unverified output. Generated tests that do not import, compile or pass cost reviewers more time than they save. Industrial pipelines filter them out [1], which is exactly the filter that discards bug-revealing tests.

Table 1: Closest systems. "Keeps bug-revealing" means tests that fail because of a suspected code bug stay in the delivered suite.

System	Lang.	Intent before tests	Bug vs. test triage	Keeps bug-revealing	Cross-test interference
ChatTester [11]	Java	Intention prompt, per run	–	–	–
TestPilot [10]	JS	–	–	–	–
TestGen-LLM [1]	Kotlin	–	–	No (filters)	Flakiness filter
CoverUp [2]	Python	–	–	No (filters)	Yes
Cover-Agent [9]	Several	–	–	No (filters)	–
Test4Py [5]	Python	Callee-first summaries	–	–	–
IGTG&R [6]	Python	From signatures, body hidden	–	–	–
TestAgent [12]	Java, Py	Stored summary	Test vs. method issue	Yes (failing)	Not reported
Code2Test	Python	Stored, scored, editable	Test, code or intent	Yes (xfail)	Yes

Failure triage. When a generated test fails, the test may be wrong, the code may be buggy, or the inferred purpose may be wrong. Each case needs a different response, and treating every failure as a broken test throws away the most useful ones.

Code2Test is designed around an explicit, persisted statement of intent per component. The intent serves as the oracle for generation and as the reference for diagnosis. This paper makes the following contributions:

1. **A tool design** that combines intent-first generation, execution-verified repair, three-way failure diagnosis and a written-suite invariant. Under the invariant, every written test passes or is an explicit expected failure that names a suspected bug (Section 3).
2. **A built-in self-evaluation** that reports, on the user's own project, the share of generated tests that pass before repair, the mutation score, and the accuracy of the failure diagnosis on failures whose cause is known (Section 3.6).
3. **An empirical study** on two libraries with two models, reporting validity, fault detection and diagnosis accuracy, and analyzing where the generated suites fall short (Section 5).
4. **Eight failure modes** that appeared only when scripted test doubles were replaced by a real model, with the fix for each (Section 5.5).

We are explicit about novelty: Section 2 shows that each technique has a precedent, and Section 6 states which parts are new. Code2Test, the experiment scripts and all raw results are open source.

2 Related Work

Table 1 compares the closest systems on the features Code2Test combines.

LLM test generation. TestPilot [10] and ChatTester [11] showed that LLMs write useful unit tests when the prompt carries signatures, documentation and execution feedback. ChatTester also

asks the model to state the focal method's intention first. CoverUp [2] targets Python. It adds coverage guidance, a tool that returns definitions with their bodies elided, up to two repair rounds, and handling of tests that pollute others. Test4Py [5] builds natural-language function summaries in callee-first order over the call graph and passes them into generation.

Keeping only passing tests. TestGen-LLM [1] keeps a candidate test only if it builds, passes five times and adds coverage. Cover-Agent [9] follows the same design. Mathews and Nagappan [7] showed that such filters reject bug-revealing tests and keep bug-validating ones. Zhao et al. [13] mitigate the effect by replacing the code under test in the prompt with a generated specification. IGTG&R [6] similarly hides the method body when inferring intent and reports detecting 67–83% of injected defects.

Deciding whether the test or the code is wrong. TestAgent [12] is the closest system. It writes and stores a structured function summary, separates test issues from focal-method issues by a three-model vote, and adds defect-revealing tests to the final suite. The authors confirmed 154 of 167 flagged tests as real bugs by manual review. Testora [8] uses a pull request's description as intent to decide whether a behavior change is intended. Meta's JiTTests [3] generate tests meant to fail on a buggy change and triage them with rule-based and LLM assessors.

Evaluating triage with known causes. Konstantinou et al. [4] crossed correct and μBERT-mutated code with correct and deliberately wrong assertions. They used the result to measure how well LLMs classify oracles, and found 52–69% accuracy that dropped when the code was buggy. Code2Test's benchmark applies the same idea inside the tool, on the user's project, to score the tool's own triage agent.

3 Approach

Figure 1 shows the pipeline. Each component, meaning a top-level function or class, passes through seven steps. The design goal is an invariant on the output: *every test Code2Test writes either passes or is an explicit expected failure that names a suspected bug.*

3.1 Intent inference

Static analysis scores each component c from signals in its source:

$$conf(c) = \min\big(0.99, \max\big(0.1,\ 0.5 + w_{\text{doc}} + w_{\text{type}} + w_{\text{name}} + w_{\text{call}} - p_{\text{cx}}\big)\big). \tag{1}$$

Here $w_{\text{doc}} \le 0.30$ grows with docstring length, $w_{\text{type}} = 0.20$ if the signature has type hints, and $w_{\text{name}} = 0.15$ for a recognized naming convention such as `validate_*` or `is_*`. The call-site weight $w_{\text{call}} \le 0.20$ grows with the number of call sites, and $p_{\text{cx}} \le 0.15$ penalizes cyclomatic complexity above 5. The intent text starts from the docstring's first sentence. Below a threshold (default 0.6), an LLM re-infers the intent from the source and the more confident result is kept. Intents are stored in SQLite. A developer can review and edit them; an edited intent receives confidence 0.95 and is never replaced by a later run.

3.2 Context and generation

The test agent's prompt contains:

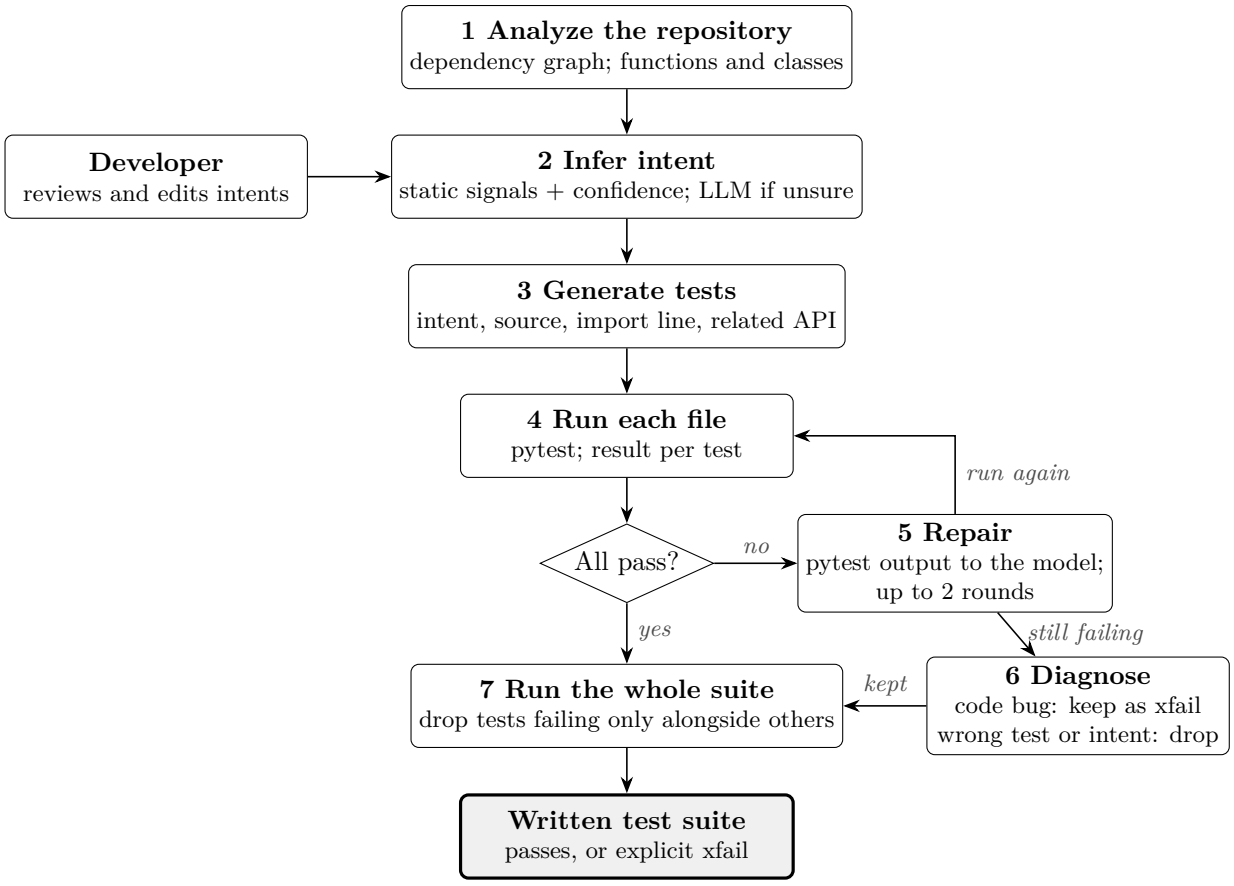


Figure 1: The Code2Test pipeline. Every component follows the main column; failing tests enter the repair loop and, if repair fails, the diagnosis branch.

- the intent and its confidence;
- the exact import line for the component;
- the imports at the top of its module, with relative imports made absolute;
- the bodiless public API (signatures, constructors, public methods) of the component's module and of the project modules it imports;
- up to 16,000 characters of the component's source.

The prompt asks the model to trace each expected value through the source, and to assert a property (type, length, membership, an exception) when an exact value is uncertain. It also asks the model to build objects exactly as their signatures require, to put any file the code creates under pytest's `tmp_path`, and never to mock the unit under test. The model returns test functions, imports and fixtures as typed structured output, and each component gets its own test file.

3.3 Execution and repair

Each file runs alone under pytest in a subprocess. The working directory is a throwaway temporary directory, with `-rootdir` at the repository, so tests that write relative paths cannot touch the user's files. Results come from pytest's JUnit XML report and are matched to the test functions each

Listing 1: A test kept because the diagnosis suspects a code bug (illustrative).

```
@pytest.mark.xfail(reason="code2test: possible bug - ordinal() returns "
                          "'rd' for 13, but 11-13 take 'th'", strict=False)
def test_ordinal_teens_take_th():
    assert ordinal(13) == "th"
```

generated test defines. An error that prevents collection, such as a bad import or a syntax error, counts against every test in the file. Failing tests go back to the model with their real pytest output, the fixtures and the related API, for up to two rounds.

3.4 Diagnosis and the expected-failure policy

Tests that still fail are classified against the stored intent:

- **Wrong test** (`TEST_WRONG`): the test misreads the intent. Errors raised by the test itself (import, syntax and name errors) always fall in this class.
- **Code bug** (`CODE_BUG`): the code contradicts the intent.
- **Wrong intent** (`INTENT_WRONG`): the inferred intent does not describe the code's purpose.

A suspected code bug is kept, decorated as in Listing 1. Everything else is dropped, and a file that cannot be collected is never kept, because `xfail` cannot cover an error at import time.

3.5 Whole-suite check

Tests that pass alone can fail together, for example when one test changes module-level state that another reads. After per-file verification, all generated files run together, and tests that fail only there are dropped, for up to three rounds.

3.6 Self-evaluation

Every run records how many components got passing tests and what share of the generated tests passed before any repair. Skipped tests do not count as passing. The `benchmark` command adds two measurements on the generated suite.

Mutation score. The benchmark enumerates single-site mutants in the tested components: flipped comparisons, swapped arithmetic and Boolean operators, changed constants, and `return None`. Docstrings and type annotations are excluded, because no test can observe them. It samples up to N mutants and runs the *whole* generated suite against each one. It also reports the share killed by the mutated component's own tests. Before mutating a file it keeps a byte-exact backup, and it restores the file on exit, including on SIGTERM; a run that was killed is repaired the next time the command starts.

Diagnosis accuracy. Failures with a known cause are passed to the diagnosis agent.

- A test of the mutated component that fails on a mutant is a `CODE_BUG` case. The mutated line must lie within the source the agent is shown.

- A passing test whose first literal expected value is perturbed is a `TEST_WRONG` case (an integer is incremented, or a string loses its last character). It counts only if the perturbed assertion is what fails.

The perturbations deliberately leave no marker in the code, such as a "_wrong" suffix, that would give the answer away.

4 Implementation

Code2Test is a Python 3.12 command-line tool (`test`, `verify`, `intent`, `benchmark`, `report`). Its dependency analysis comes from the CodeWiki analyzer (tree-sitter, several languages); only Python components get tests in the current version. The three agents use pydantic-ai with typed structured output and up to three output retries. They work with any model pydantic-ai supports, including OpenAI-compatible endpoints. HTTP 401, 403 and 404 responses stop the run, so a bad key or model name cannot produce an apparently successful run with no tests. The tool's own test suite (over 100 tests) drives the whole pipeline offline with scripted models.

5 Evaluation

We answer four research questions.

RQ1 How many components end up with passing tests, and how many generated tests pass before any repair?

RQ2 Do the written suites detect injected faults?

RQ3 How accurate is the failure diagnosis on failures with a known cause?

RQ4 What breaks when scripted test doubles are replaced by a real model?

5.1 Setup

Subjects. We used two pure-Python libraries from PyPI, after removing each library's own tests:

- **inflection 0.5.1**: 426 lines and 13 components; string functions over module-level rule tables.
- **TinyDB 4.8.2**: 2,170 lines and 26 components; classes with state, JSON file storage, caching middleware and a query language.

Models. MiniMax `MiniMax-M3.1-Flash-Preview` ("Flash") and `MiniMax-M3` ("M3"), accessed through MiniMax's OpenAI-compatible API with default settings.

Procedure. For each subject and model, one run of `code2test test -auto`, then `code2test verify`, then `code2test benchmark` with 80 sampled mutants and 15 diagnosis cases per expected cause. The MVP targets come from the project proposal: at least 70% of components with tests, at least 80% of tests passing before repair, and at least 75% diagnosis accuracy.

5.2 RQ1: Acceptance and first-run validity

Table 2 shows that every run met both generation targets. Each written suite passed `verify`; the one TinyDB case not counted as passing was skipped by its own test, not failed. The single TinyDB component without tests differed by model. With Flash, the generation call for `LRUCache` failed

Table 2: Results per subject and model (one run each). Test cases count parametrized cases separately. Mutants are sampled from the components that have tests, so the two TinyDB runs use different samples.

	inflection		TinyDB	
Measure	Flash	M3	Flash	M3
Components with passing tests	13/13	13/13	25/26	25/26
acceptance rate (target 70%)	100%	100%	96%	96%
Tests generated	205	172	471	297
passing before repair (target 80%)	96%	85%	87%	99%
Tests kept / dropped	205/0	172/0	458/13	296/1
Tests kept as xfail	0	0	0	0
Repair rounds	8	5	22	4
Test cases passing in `verify`	408/408	326/326	530/531	432/432
Time to generate and repair (s)	193	407	314	815
Mutants killed, whole suite	47/80	58/80	62/80	61/80
mutation score	59%	73%	78%	76%
by the component's own tests	50%	65%	76%	76%
Diagnosis correct (target 75%)	97%	100%	100%	100%
code-bug cases	14/15	15/15	15/15	15/15
wrong-test cases	15/15	15/15	15/15	15/15

after its retries. With M3, the model wrote a single test for the `TinyDB` class. That test still failed after two repair rounds, was diagnosed as a wrong test, and was dropped.

The stronger model did not dominate. M3 passed more tests before repair than Flash on TinyDB (99% vs. 87%) but fewer on inflection (85% vs. 96%). It also wrote fewer tests in every case.

No test was kept as an expected failure in any run, and every diagnosis made during generation was a wrong test. On two mature libraries this is the desired outcome: the expected-failure path never accused correct code. It also means the expected-failure path was not exercised by real bugs in this study.

Where the intents came from. Both libraries document nearly every component. No inflection component and 2 of the 26 TinyDB components scored below the 0.6 threshold, so the model re-inferred 2 intents per TinyDB run and none for inflection. Every other intent came from static signals, almost always the docstring's first sentence. On these subjects the intent stage therefore added little beyond the docstring, and the results do not isolate its effect.

5.3 RQ2: Fault detection

On whole-suite mutation analysis, the suites killed between 59% and 78% of the sampled mutants (Table 2). Tests of other components added few kills: the whole suite killed 47, 58, 62 and 61 mutants, against 40, 52, 61 and 61 for the mutated component's own tests. The survivors are concentrated rather than spread out.

inflection. 40 of the 80 sampled mutants fall in `_irregular`, which builds the rules for irregular words such as *person/people* and *cow/kine* when the module is imported. Flash wrote a single test for it, which checks how many rules are inserted, and no Flash test checks an irregular word through `pluralize` or `singularize`. That suite killed 9 of the 40 `_irregular` mutants. M3 wrote

19 tests for `_irregular` that check words whose singular and plural share a first letter, such as *person/people*. It killed 18; its survivors are in the branch for words whose first letters differ, such as *cow/kine*. Outside `_irregular`, the suites killed 38 of 40 (Flash) and 40 of 40 (M3) mutants. Passing tests can thus check structure rather than behavior, and whether a run tests a behavior at all varies between runs (Section 6.3).

TinyDB. 14 of 18 (Flash) and 12 of 19 (M3) surviving mutants change a string literal. These are the operator names that `Query` uses as hash keys, exception messages and `repr` text. Tests that check an exception's type but not its message cannot see such changes, and many hash-key changes are likely equivalent mutants, because a key only needs to differ from the others.

5.4 RQ3: Diagnosis accuracy

On failures of known cause, the diagnosis agent was correct in 97%, 100%, 100% and 100% of cases across the four runs, above the 75% target. It made one error across the four runs. In inflection with Flash, a mutant sent `_irregular` down its other branch, so the test's expected rule counts no longer held. The agent read the mutated source and judged the test wrong, because the code now inserted more rules than the test expected. The intent ("a convenience function to add appropriate rules to plurals and singular for irregular words") says nothing about the number of rules, so the agent sided with the code. This is the misguidance effect [13] in miniature: where the intent is silent, the code decides. The cases are synthetic, and the threats below explain why they are likely easier than real failures.

5.5 RQ4: What a real model broke

During development, the tool's own tests used scripted models and passed throughout. The first real-model runs exposed the eight failure modes in Table 3. Each was fixed with a regression test before the results above were measured. On inflection the written suite went from failing to collect (first real run), to 676 of 680 test cases passing (second run), to passing in full from the third run on.

6 Discussion

6.1 What is new

Table 4 states, claim by claim, what has a precedent. None of Code2Test's techniques is new on its own. The closest system, TestAgent [12], shares the stored summary, the bug-versus-test triage and the retention of defect-revealing tests, and it was evaluated on larger Java and Python projects. What Code2Test adds is a specific combination:

- an intent that is scored and developer-editable, and reused across runs and in triage;
- a third diagnosis class for a wrong intent;
- retention of suspected bugs as expected failures, so the delivered suite stays green;
- self-evaluation, including triage accuracy, on the user's own project.

We therefore present Code2Test as a tool and experience report rather than a new technique.

Table 3: Failure modes found only with a real model, and their fixes.

Failure mode	Symptom	Fix
Rejected API key looked like success	Every call returned HTTP 401, yet the run exited 0 with no tests	Authentication and unknown-model errors stop the run
Nested functions treated as components	inflection's `caseinsensitive` lives inside `_irregular`; its tests could never import it	Skip functions nested in functions
Unimportable files kept as xfail	xfail cannot cover a module-level import error; the whole suite failed to collect	Never keep a file that cannot be collected; errors raised by the test count as a wrong test
Reported test names differ from defined names	Failures matched no test, so failing files were kept unverified	Match results on the functions each test defines
Doubly escaped code	Literal \n and \" in code or fixtures broke whole files	Unescape only when that makes the code compile
Tests interfering across files	Tests registering irregular words broke `pluralize` tests in the combined run	Whole-suite check after per-file verification
Missing API context	Tests passed an instance where TinyDB expects a storage class; 0 of 6 passed	Related project API in the prompt; 43 of 43 passed
Tests writing into the repository	An unrepaired test created `some/path` in the project directory	Run pytest from a temporary directory; prompt requires `tmp_path`

6.2 The open question

The design is motivated by the misguidance effect [7, 13], but this paper does not yet show that Code2Test avoids it. Both subjects are mature and, as far as we know, correct. The decisive experiment uses buggy programs with known fixes, such as Refactory as in Mathews and Nagappan, and BugsInPy, and compares Code2Test with CoverUp, Cover-Agent and TestAgent under the same model. For each tool it would count tests that pass on the buggy version and fail on the fix (bugs locked in) against tests that flag the bug, with and without intent-first generation. That study is the natural next step.

6.3 Threats to validity

- **Scale and variance.** Two small libraries and one run per configuration, so the results carry no estimate of run-to-run variance, and the variance we saw during development is large. In a development run with Flash and an earlier version of the tool, the inflection suite had 33 tests for `_irregular`, several checking irregular words through `pluralize` and `singularize`, and killed all 80 mutants of its own sample even when only each component's own tests ran. Across development runs, TinyDB's mutation score with Flash ranged from 64% to 79% under that per-component scoring.
- **Mutants.** The 80 mutants are sampled uniformly from all mutation sites, so large functions dominate the sample: half of inflection's sample falls in `_irregular`. There is no filter for equivalent mutants.
- **Synthetic triage labels.** Mutated code is a code bug by construction, and the intent was

Table 4: Novelty of each claim, judged against the closest prior work.

Claim	Closest prior work	Verdict
Infer intent before writing tests	ChatTester, Test4Py, TestAgent, IGTG&R, Zhao et al.	Incremental
Classify failures as test, code or intent	TestAgent, Testora, JiTTests	Incremental
Keep bug-revealing tests in the suite	TestAgent (as failing tests)	Incremental
Repair, discard, cross-test interference	CoverUp, TestGen-LLM	Not new
Bodiless API context	CoverUp	Not new
Dependency-ordered summaries	Test4Py	Not new
Triage accuracy from failures of known cause	Konstantinou et al.	Not new as a method
Real-model failure modes; two-model comparison	–	Empirical

inferred from the original code. Wrong-test cases differ from a passing test by one perturbed literal. Both make the cases easier to classify than real failures.

- **Training data.** Both libraries are public and may appear in the models' training data.
- **Same authors.** The tool and the benchmark come from the same authors. An early benchmark version left markers in its injected faults that overstated diagnosis accuracy until it was fixed. Two metric definitions were also corrected during development: skipped tests no longer count as passing, and re-runs no longer report 0% acceptance. All reported numbers use the corrected definitions.

6.4 Limitations

Only Python components get tests. An expected-failure test is only as good as the diagnosis behind it: a wrong code-bug verdict keeps a wrong test as an expected failure. Code that needs databases, networks or other services gets tests only when the model can build fixtures for it. Each run sends source code to the model provider and costs one or more model calls per component.

7 Conclusion

Code2Test infers and stores an explicit intent for each component of a legacy Python codebase. It then generates, executes, repairs and diagnoses tests against that intent, and keeps suspected bugs visible as expected failures rather than filtering them out. On two libraries and two models it produced passing tests for nearly every component, and at least 85% of generated tests passed before any repair. The suites killed 59%–78% of sampled mutants; most survivors sat in behavior a run had tested only structurally, or changed strings that tests do not observe. The diagnosis was correct on all but one of the failures of known cause. Its techniques have precedents; its value lies in the combination, the built-in self-evaluation, and the failure modes we document. Whether intent-first generation avoids locking bugs into generated suites is the question we plan to answer next.

Data Availability

The tool, the experiment scripts and all raw results are available at `https://github.com/Cirkitly/code2test`, under `research/`. For each run, the results include the generated test suite, the inferred intents, the run metrics, the benchmark output with every surviving mutant and diagnosis case, and the verification log. `research/scripts/run_experiments.sh` reproduces the runs, given a MiniMax API key.

References

[1] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. Automated unit test improvement using large language models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024. URL `https://arxiv.org/abs/2402.09171`.

[2] Juan Altmayer Pizzorno and Emery D. Berger. CoverUp: Effective high coverage test generation for Python. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025. doi: 10.1145/3729398. URL `https://arxiv.org/abs/2403.16218`.

[3] Matthew Becker, Yifei Chen, Nicholas Cochran, Pouyan Ghasemi, Abhishek Gulati, Mark Harman, Zachary Haluza, Mehrdad Honarkhah, Herve Robert, Jiacheng Liu, Weini Liu, Sreeja Thummala, Xiaoning Yang, Rui Xin, and Sophie Zeng. Just-in-time catching test generation at Meta, 2026. URL `https://arxiv.org/abs/2601.22832`.

[4] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. Can we find bugs using LLM-generated oracles?, 2024. URL `https://arxiv.org/abs/2410.21136`. Revised 2026.

[5] Runlin Liu, Zhe Zhang, Yunge Hu, Yuhang Lin, Xiang Gao, and Hailong Sun. Type-aware LLM-based regression test generation for Python programs, 2025. URL `https://arxiv.org/abs/2503.14000`.

[6] Xiaojian Liu and Yangyang Zhang. IGTG&R: An intent analysis-guided unit test generation and refinement framework. *Entropy*, 28(1):74, 2026. doi: 10.3390/e28010074.

[7] Noble Saji Mathews and Meiyappan Nagappan. Design choices made by LLM-based test generators prevent them from finding bugs, 2024. URL `https://arxiv.org/abs/2412.14137`.

[8] Michael Pradel. Testora: Using natural language intent to detect behavioral regressions. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. URL `https://arxiv.org/abs/2503.18597`.

[9] Qodo. Cover-Agent: An AI-powered tool for automated test generation and code coverage enhancement. `https://github.com/qodo-ai/qodo-cover`, 2024.

[10] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024. URL `https://arxiv.org/abs/2302.06527`.

[11] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. Evaluating and improving ChatGPT for unit test generation. In *Proceedings of the*

ACM International Conference on the Foundations of Software Engineering (FSE), 2024. URL https://arxiv.org/abs/2305.04207. arXiv:2305.04207.

[12] Quanjun Zhang, Ye Shang, Siqi Gu, Jianyi Zhou, Chunrong Fang, Zhenyu Chen, and Liang Xiao. Multi-agent LLM collaboration for unit test generation via human-testing-inspired workflows, 2026. URL https://arxiv.org/abs/2607.09101.

[13] Junda Zhao, Shurui Zhou, and Eldan Cohen. Evaluating and mitigating the misguidance effect of buggy code in LLM-generated unit tests. *Proceedings of the ACM on Software Engineering*, 3(ISSTA), 2026. URL https://arxiv.org/abs/2607.22883.